

component communication is the mechanism by which data and events flow between components. Since Angular follows a **component-based architecture**, understanding communication patterns is critical for building scalable applications.

◆ 1. Why Component Communication Matters

Angular applications are composed of **hierarchies of components**:

- Parent → Child → Grandchild

Each component is **isolated**, so data sharing must be explicit.

You typically need communication for:

- Passing data **downward**
 - Sending events **upward**
 - Sharing state between **unrelated components**
-

◆ 2. Types of Component Communication

There are **4 primary patterns**:

✅ 1. Parent → Child Communication (@Input())

Used when a parent needs to **send data to a child component**.

📌 How it works:

- Parent binds data
- Child receives via @Input()

💻 Example

Child Component

```
import { Component, Input } from '@angular/core';
```

```
@Component({
```

```
    selector: 'app-child',
    template: `<p>Message: {{ message }}</p>`
  })
export class ChildComponent {
  @Input() message!: string;
}
```

Parent Component

```
<app-child [message]="Hello from Parent"></app-child>
```

Key Points

- One-way data flow
- Uses **property binding**
- Runs during change detection

2. Child → Parent Communication (@Output() + EventEmitter)

Used when a child needs to **send data/events to the parent**.

How it works:

- Child emits event
- Parent listens

Example

Child Component

```
import { Component, Output, EventEmitter } from '@angular/core';
```

```
@Component({
  selector: 'app-child',
  template: `<button (click)="sendData()">Send</button>`
})
```

```
export class ChildComponent {  
  @Output() notify = new EventEmitter<string>();  
  
  sendData() {  
    this.notify.emit('Hello Parent!');  
  }  
}
```

Parent Component

```
<app-child (notify)="receiveData($event)"></app-child>  
  
receiveData(data: string) {  
  console.log(data);  
}
```

Key Points

- Event-driven communication
- Uses Angular event binding
- Common in forms, button actions

3. Sibling / Unrelated Components (Using Service + RxJS)

When components are **not directly related**, use a **shared service**.

This is the **most important real-world pattern**.

How it works:

- Service acts as a mediator
- Uses RxJS (Subject, BehaviorSubject)

Example

Shared Service

```
import { Injectable } from '@angular/core';
```

```
import { Subject } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class DataService {

  private messageSource = new Subject<string>();
  message$ = this.messageSource.asObservable();

  sendMessage(msg: string) {
    this.messageSource.next(msg);
  }
}
```

Sender Component

```
constructor(private dataService: DataService) {}

send() {
  this.dataService.sendMessage('Hello from Sender');
}
```

Receiver Component

```
constructor(private dataService: DataService) {}

ngOnInit() {
  this.dataService.message$.subscribe(msg => {
    console.log(msg);
  });
}
```

Key Points

- Decoupled architecture
 - Uses **Reactive Programming (RxJS)**
 - Preferred for large apps
-

✅ 4. ViewChild / Parent Accessing Child Directly

Used when parent needs **direct access to child methods or properties**.

📌 How it works:

- Parent gets reference using `@ViewChild`

💻 Example

Child Component

```
export class ChildComponent {  
  sayHello() {  
    console.log('Hello from Child');  
  }  
}
```

Parent Component

```
import { ViewChild, AfterViewInit } from '@angular/core';  
import { ChildComponent } from './child.component';  
  
export class ParentComponent implements AfterViewInit {  
  
  @ViewChild(ChildComponent) child!: ChildComponent;  
  
  ngAfterViewInit() {  
    this.child.sayHello();  
  }  
}
```

```
}
```

Key Points

- Tight coupling ⚠️
- Should be used sparingly
- Useful for DOM or imperative logic

◆ 4. Summary Table

Scenario	Technique	Coupling	Usage
Parent → Child	@Input()	Loose	Data passing
Child → Parent	@Output()	Loose	Events
Siblings	Service + RxJS	Very Loose	Shared state
Parent → Child (direct)	@ViewChild()	Tight	Method access
